What is algorithm

" An algorithm is a finite state of instructions if follows accomplishes a particular task

CRITERIA OF ALGORITHM

INPUT - 0 or more quantity is produced

Definiteness - each instructions is clear and unambiguos

finiteness - the algorithm shoot terminate after a finite no. of steps.

effectiveness - every instructions must be very basics so that it can be carried out

program - program is the expression of an algorithm in a programing language sometime words such as procedure, function, and subroutine are used synonymd for program.

Analysis of Algorithm.

Systematic evolution of how algorithm behaves concerning time complexity space complexity, and practical efficiency while manipulating different types of data

What is important then performance

1. Modularity
2. Correctness
3. Maintainability
4. functionality
5. Robustness
6. User friendlyness
7. guess program time
8. Simplicity
9. Extensiblity
10. Elaborating.

Asymtotic Notation.

Notation.

• worst case	• Average case	• Best case
• O (Big-O notation)	• θ (theta notation)	• Ω (omega notation
• lower Bound	• Average	• Upper bound

Time complexity.

" It deals with the quantification of the amount of time taken by set of Code or algorithm to process or run as a function of the amount of input.

Scanned with OKEN Scanner

Defination

"time complexity is define as the no. of time a particular instructions set is executed rather than total time taken It depends on some external factors like compiler used, the processor speed etc. the time T(P) taken by a program is sum of compiler time & run time. (execution time)

1) Constant time O(1) (worst case)

It is a time complexity of an algorithm that always uses the same no. of operation regardless of no. of element been operated on O(1) does not imply that only one operation is used but rather that the no. of operation is always constant

2) linear type O(n)

it indicates that the time it takes to run and algorithm grows in linear passion as (n) increases.

In linear type searching a list of a thousand records should take roughly 10 times as long as searching a list of 100 records.

Scanned with OKEN Scanner

Page No.
Date

Quadratic time $O(n^2)$
it indicates that the time needed to run an algorithm grows as (n^2)

logarithmic time $O(\log n)$
It indicates that the time needed to run a algorithm grows as logarithm of n
eg: binary search on a sorted list is performed, the list is search by dividing it in half repeatedly until the desired element is found.
The no. of divisions neccessary to find the element grows with the algorithm n in base to rather than proportionality to n
$(n \log n)$ is lower growth rate then O (~~log~~ $\log n$) this algorithm have lower time complexity. than linear time algorithm.

Scanned with OKEN Scanner

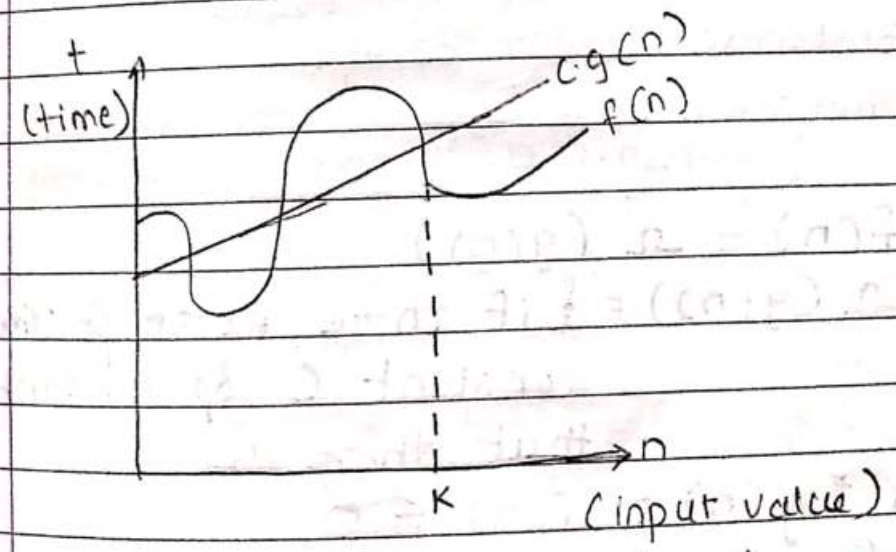
Asymtotic notation.

The efficiency of an algorithm depends on the amount of time, storage and other resources required to execute the program it is measure with the help of a symptotic notation.

An algorithm may not have the same performance for different types of input

With the increase in the input size the performance will change

1) Big-oh(O) notation



worst case upper band

Maximum time required for execution

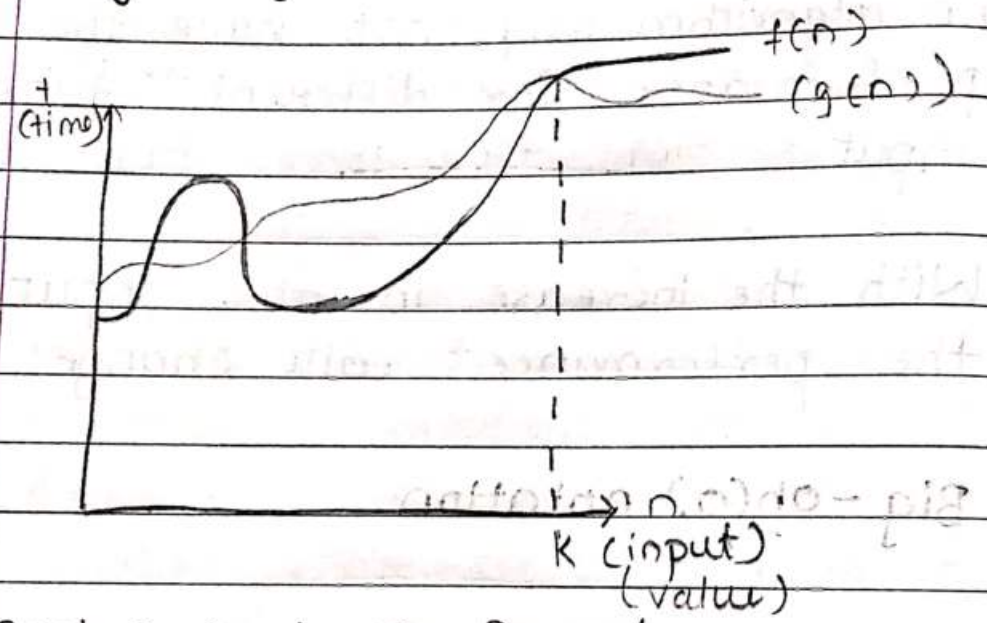
$f(n) = O(g(n))$

$O(g(n)) = \{$ if there exist positive constant c & k such that there is

$0 \leq f(n) \leq c * g(n)$

for all $n \geq k$

2) Big Omega (Ω)



Best case lower Bound

minimum time required for execution.

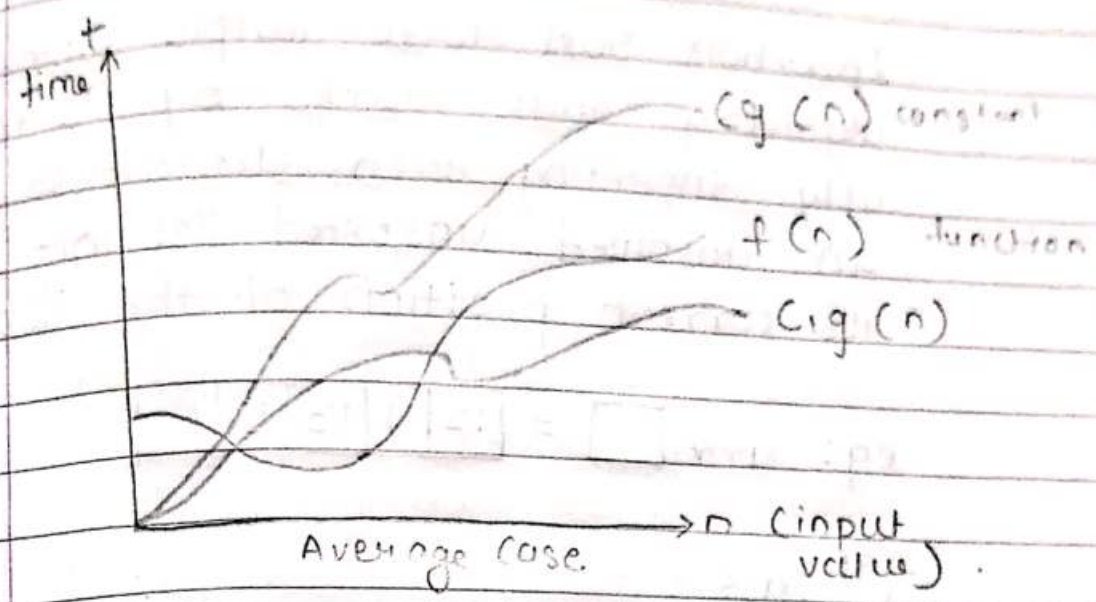
$f(n) = \Omega(g(n))$

$\Omega(g(n)) = \{$ if there exist positive constant c & k, such that there is

$c * g(n) \leq f(n) \leq 0$

for all $n \geq k$

3) theta (θ)



Average time required for execution

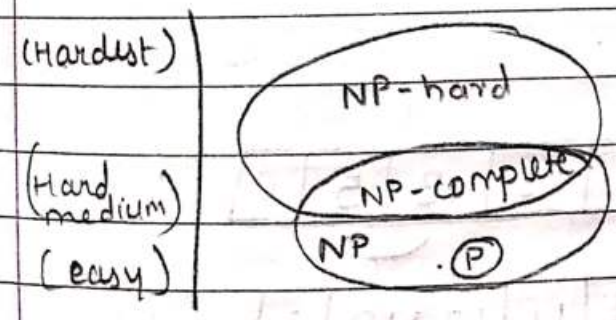
$F(n) = \theta(g(n))$

$\theta(g(n)) = \{$ if there exist positive constant C, C_1 & K such that there is

$0 \leq (g(n) \leq f(n) \leq C_1 g(n)$

for all $n \geq k$.

NP, NP-complete, P, NP-hard



Easy - P

Medium - NP

Hard - NP complete

Hardest : NP hard.

Insertion Sort Algorithm

Insertion Sort is a simple sorting algorithm that works by itteratively inserting each element of an unsolved unsorted list into its correct position of the list

eg: arr [ ] = | 12 | 11 | 13 | 5 | 6 |

1st pass

arr [ ] = | 12 | 11 | 13 | 5 | 6 |

arr [ ] = | 11 | 12 | 13 | 5 | 6 |

2nd pass

arr [ ] = | 11 | 12 | 13 | 5 | 6 |

arr [ ] = | 11 | 12 | 13 | 5 | 6 |

3rd pass

arr [ ] = | 11 | 12 | 13 | 5 | 6 |

arr [ ] = | 5 | 11 | 12 | 13 | 6 |

Scanned with OKEN Scanner

4th pass

arr [ ] = | 5 | 11 | 12 | 13 | 6 |

arr [ ] = | 5 | 6 | 11 | 12 | 13 |

Q Initial Sort Algorithm

→ initially : | 23 | 1 | 10 | 5 | 2 |

pass 1 : | 1 | 23 | 10 | 5 | 2 |

pass 2 : | 1 | 10 | 23 | 5 | 2 |

pass 3 : | 1 | 5 | 10 | 23 | 2 |

pass 4 : | 1 | 2 | 5 | 10 | 23 |

explanation :

- Current element is 23
  the first element in array is assume to be sorted the sorted part until 0 index is : [23]

- Now the 1st pass
  Compare 1 with 23 since 1 is smaller insert 1 before 23 the sorted part [1, 23] until 1st index is : [1, 23]

Scanned with OKEN Scanner

- Now the 2nd pass
Compare 10 with 23 since 10 is greater than 1 and smaller than 23 insert 10 before 23 [1, 10, 23] the sorted part until 2nd index is : [1, 10, 23]

- Now 3rd pass
Compare 5 with 1, 10 & 23 Since 5 is greater than 1 & smaller than 10 insert 5 between 1 & 10 the sorted part until 3rd index is : [1, 5, 10, 23]

- Now 4th pass
Compare 2 with 1, 5, 10, & 23 Since 2 is greater than 1 & smaller than 5 insert 5 between 1 & 5 the sorted part until 4th index is : [1, 2, 5, 10, 23]

- Now final array
the sorted array is 1, 2, 5, 10, 23 : [1, 2, 5, 10, 23]

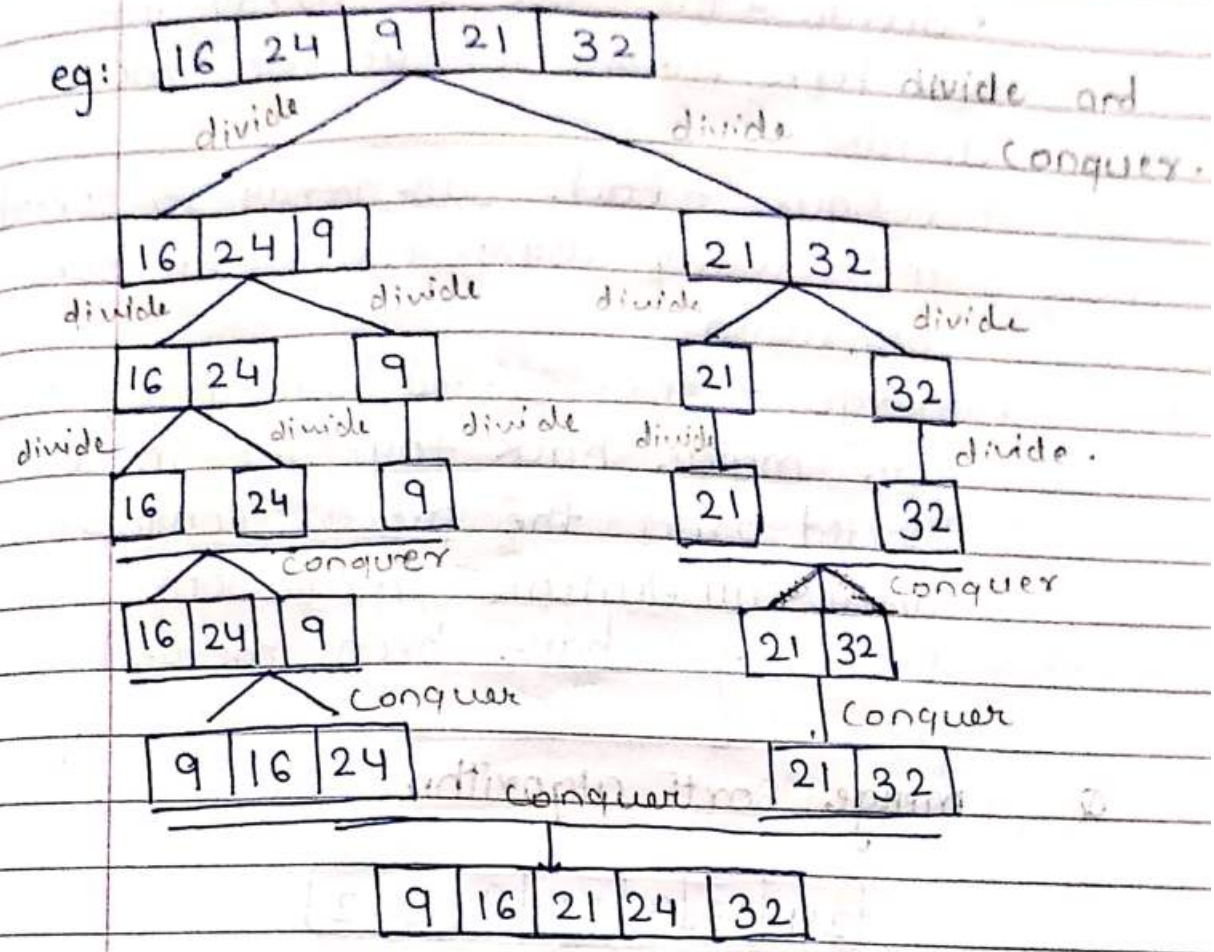
Best case $O(n)$
worst case $O(n^2)$
Average case $O(n^2)$
} time complexity

Scanned with OKEN Scanner

MERGE SORT ALGORITHM

Q Merge Sort Algorithm

eg:

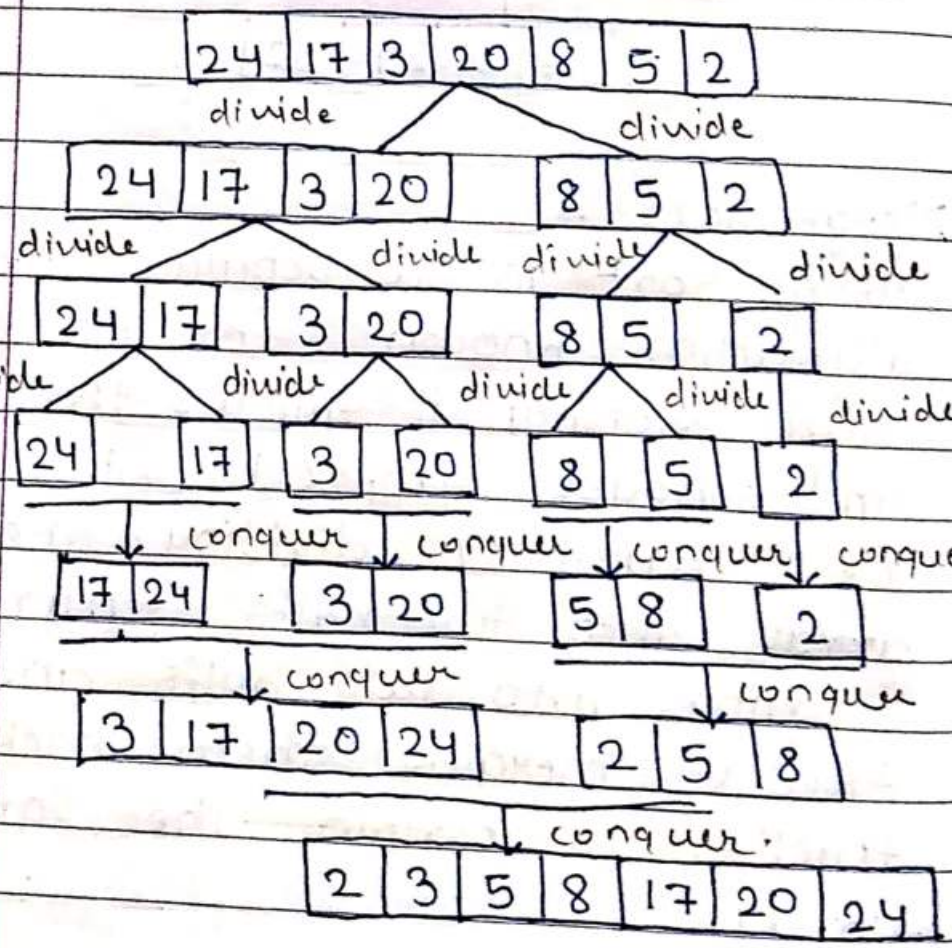


- Merge Sort

Merge sort is a popular sorting algorithm known for its efficiency and stability it follows the divide and conquer approch it works by recursively dividing the input array into two halfs recursively sorting into two halfs and finally merging them back together to obtain the sorted array

- Working
  - divide - the list or array into two halfs untill it can no more divided
  - Conquer - each sub-array is sorted individually using the merge sort algorithm
  - merge - the sorted sub-arrays are merge back together in a sorted order the process continues untill all elements from both sub-arrays have been merged

Q merge sort algorithm



Quick Sort

Quick Sort is a sorting algorithm based on quick divide and conquer approch that picks an element as a pivot and partitions the given array arround the picked pivot in it's correct position in a sorted array

- Working
- Choose a pivot - select an element from the array as a pivot. the choice of pivot can vary (eg: 1st element, last element, random element or middle element)
- partition the array -
- (rearranged the array) - around the pivot after partitioning all elements smaller than the pivot will be on its left, and all elements greater than the pivot will be on its right.

the pivot is then in its correct position, and we obtain the index of the pivot

- recursively call - recursively apply the same process to the two partition sub-arrays (left, right

of the pivot)

eg:

[0]	[1]	[2]	[3]	[4]
10	80	30	90	40

Step 1st : pivot selection - the last arr [4] = 40 is choosen as the pivot

initial pointers : i = -1 & j = 0

arr [] =

j				Pivot
10	80	30	90	40

compare all with pivot.

Step 1:

arr[] =

j				
10	80	30	90	40
0	1	2	3	4

i = -1

Step 2:

10	80	30	90	40

10	80	30	90	40

Step 3:

j

10	80	30	90	40

i

10	80	30	90	40

Step 4:

i j

10	80	30	90	40

10	30	80	90	40

Scanned with OKEN Scanner

Step 5:

		i	j	
10	30	80	90	40

10	30	80	90	40

Step 6:

		i		j
10	30	80	90	40

now compare with pivot 40.

10	30	40	90	80

Step 7:

10	30	40	90	80

10	30	80	90

as 90 is greater than 80 & & 80 is greater than 40 & smaller than 90. Hence 80 is placed between 40 & 90

10	30	40	80	90

Scanned with OKEN Scanner